Table of Contents

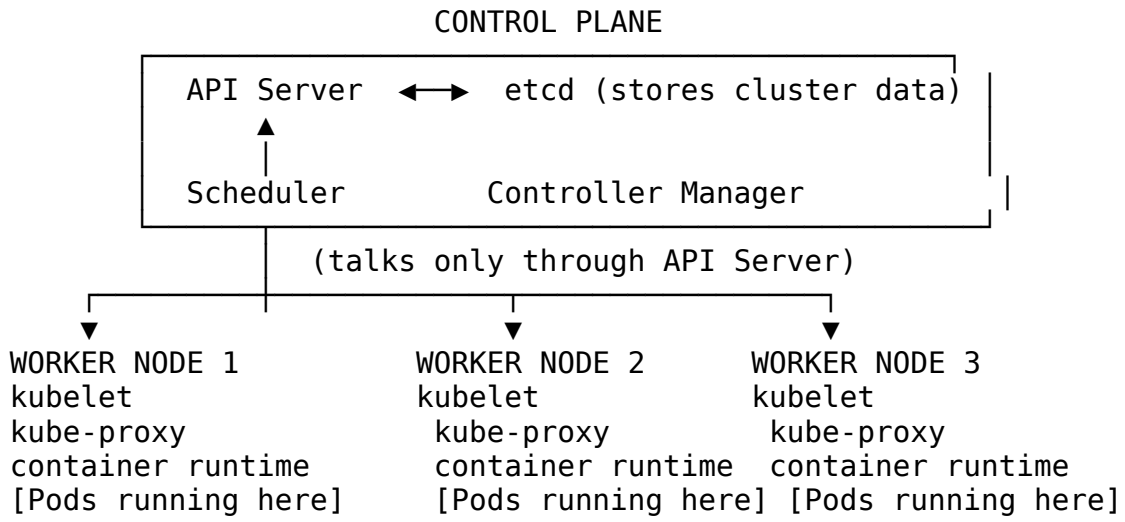
Kubernetes Notebook — Technical Notes (Plain English)

This version keeps all the technical terms and depth, but explains each mechanism step-by-step in plain, everyday English rather than dense textbook language.

Chunk 1: Core Architecture & CLI

1. Kubernetes Architecture

A Kubernetes cluster has two kinds of machines: **Control Plane nodes** (the decision-makers) and **Worker nodes** (the machines that actually run your applications). The Control Plane never runs your application code — it only tracks what should be running and tells workers what to do.



Here's the important rule to remember: **every single component in the cluster — the Scheduler, the Controller Manager, every kubelet on every worker — only ever talks to the API Server.** No component talks directly to etcd except the API Server itself, and no component talks directly to another component. This "everyone talks to one central point" design makes the system predictable and easier to secure.

2. Master Node Components

kube-apiserver — This is the single entry point for every request into the cluster, whether it comes from you (via `kubectl`) or from an internal component. When a request arrives, it goes through four checks in order:

```
Request arrives
   → 1. Authentication: who are you? (checks your certificate or
token)
   → 2. Authorization: are you allowed to do this? (checks RBAC rules)
   → 3. Admission Control: should this request be modified or blocked?
        (e.g., auto-inject defaults, block disallowed configurations)
   → 4. Validation: does this request make sense structurally?
   → Saved to etcd
```

The API Server itself doesn't decide *what* should happen (that's other components' job) — it's purely the gatekeeper and record-keeper.

etcd — A key-value database that stores the entire desired and current state of the cluster: every Pod definition, every Service, every Secret. It's the single source of truth. Because it's this important, etcd runs on multiple machines (usually 3 or 5) and uses a consensus algorithm called Raft to make sure all copies agree — even if one or two machines go down, the data stays consistent. This means etcd can survive the loss of up to `(n-1)/2` of its machines without losing data.

kube-scheduler — When a new Pod is created but hasn't been assigned to a machine yet, the Scheduler picks one. It does this in two steps:

1. **Filtering** — throw out any Node that *can't* run this Pod (not enough CPU/memory, wrong taints, doesn't match required affinity rules).
2. **Scoring** — of the Nodes that are left, rank them (e.g., prefer the Node with the most free capacity) and pick the winner.

The Scheduler doesn't actually place the Pod itself — it just writes its decision back to the API Server, and the target Node's kubelet picks it up from there.

kube-controller-manager — Runs many small automated loops, each one responsible for one thing, constantly comparing "what should be happening" against "what is actually happening," and correcting any difference it finds. For example, the Node Controller checks if a Node has stopped sending heartbeats, and if so, marks it as unhealthy. This pattern — watch, compare, correct — is called a **reconciliation loop**, and it's the core idea behind almost everything in Kubernetes.

3. Worker Node Components

kubelet — The agent that runs on every worker machine. It receives instructions from the API Server about which Pods should run on its machine, then makes that happen by talking to the container runtime. It also reports back constantly: "here's what's actually running, here's the health status." Think of it as the local manager that both takes orders and reports status.

kube-proxy — Handles network traffic routing on each Node. When a request comes in for a Service (a stable address), kube-proxy is responsible for making sure that request actually reaches one of the real Pods behind it — even though Pods' individual IP addresses keep changing. It does this by writing low-level networking rules (using `iptables` or the faster `IPVS` mode) on the machine.

Container Runtime — The software that actually starts and stops containers. Kubernetes doesn't run containers itself — it delegates that job through a standard interface called CRI (Container Runtime Interface) to something like `containerd`.

Pods — The smallest thing you can deploy in Kubernetes. Usually one Pod = one application container, but a Pod can hold more than one container if they need to be tightly coupled (sharing the same network address and, optionally, files).

4. kubectl Cheat Sheet

```
# CREATE
kubectl apply -f deployment.yaml
kubectl create namespace dev
kubectl create configmap app-cm --from-literal=ENV=prod

# READ
kubectl get pods -o wide
kubectl describe pod <pod-name>
kubectl logs <pod-name> -f
kubectl top pod

# UPDATE
kubectl set image deployment/nginx nginx=nginx:1.26
kubectl scale deployment nginx --replicas=5
kubectl rollout restart deployment nginx

# DELETE
kubectl delete pod <pod-name>
kubectl delete -f deployment.yaml
```

Chunk 2: AWS EKS Specifics & Initial Constructs

1. AWS EKS Specifics

ENI Integration (Pod Networking) — Normal Kubernetes often hides Pods behind a virtual, made-up network. EKS does it differently: it uses the **AWS VPC CNI plugin**, which gives every single Pod a real, routable IP address straight from your AWS network (VPC). This is convenient — Pods can be reached directly, security groups work naturally — but it has a practical consequence: **the number of Pods you can run on a machine is limited by how many IP addresses AWS can attach to it**, not just by CPU or memory.

```
Worker Node (EC2 instance)
  ├─ Network Card 1 (Primary) → the Node's own IP
  ├─ Network Card 2
  │     ├─ IP → Pod A
  │     ├─ IP → Pod B
  │     └─ IP → (spare, ready for next Pod)
```

Fargate Profile — Lets you run Pods without provisioning or managing any EC2 worker machines at all. You define a Fargate Profile (which namespace and/or labels it applies to), and any matching Pod automatically runs on its own isolated micro-virtual-machine, managed entirely by AWS. The trade-off: Fargate can't run everything — no DaemonSets, no privileged containers, no direct host networking, and no EBS volumes (only EFS).

eksctl — A dedicated command-line tool (separate from `kubectl`) purpose-built for creating and managing entire EKS clusters, including the underlying AWS infrastructure (VPC, worker nodes, IAM roles).

```
eksctl create cluster --name my-cluster --region ap-south-1 \
  --nodegroup-name workers --node-type t3.medium --nodes 3 --managed

eksctl create fargateprofile --cluster my-cluster --name fp-default \
  --namespace dev

aws eks update-kubeconfig --name my-cluster --region ap-south-1
```

2. Namespaces

A Namespace splits one physical cluster into multiple logical sections, so different teams or environments (dev, staging, prod) don't collide with each other or step on each other's resource usage. Note that this is *logical* isolation only — Pods in different Namespaces can still talk to each other over the network unless you explicitly block that with a NetworkPolicy.

```
kubectl create namespace payments
kubectl config set-context --current --namespace=payments
```

You can also cap how much CPU/memory/Pod-count a Namespace is allowed to use with a `ResourceQuota`, which is how teams get kept from accidentally hogging the whole cluster.

3. Init Containers

Before your main application container starts, you can define one or more **Init Containers** that must run and finish successfully first, one after another (not at the same time). If any Init Container fails, Kubernetes retries the whole Pod according to its restart policy — your main app never gets a chance to start until every Init Container has succeeded.

```
[Init Container 1 runs] → finishes → [Init Container 2 runs] →
finishes → [Main app container starts]
```

This is commonly used for things like: waiting until a database is reachable, pulling configuration/secrets from an external system, or preparing files on a shared volume before the main app touches them. Init Containers don't support liveness/readiness probes because they're expected to run once and exit — they're not long-running processes.

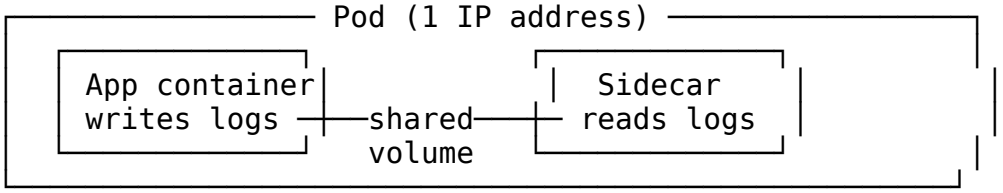
Chunk 3: The Pod Lifecycle & Workloads

1. Pod Deep Dive

A Pod bundles one or more containers that need to run together as a single unit. All containers inside one Pod share two things automatically:

- **Network namespace** — they all share one IP address and can reach each other over `localhost`, just like two processes running on the same physical machine.
- **Storage (if you set it up)** — you can attach a shared Volume that multiple containers in the Pod can read/write to, like a shared folder.

The most common pattern beyond a single container is the **sidecar** pattern — your main app container runs alongside a small helper container (for example, a logging agent that reads files your main app writes).



2. Static Pods

Normally, the Scheduler decides which Node a Pod runs on. **Static Pods break that rule** — they're defined by a file sitting directly on one specific machine's disk, and that machine's kubelet starts them itself, without asking the Scheduler or even the API Server for permission.

Why does this matter? Because it's how Kubernetes solves a chicken-and-egg problem: the Control Plane's own components (`kube-apiserver`, `etcd`, `kube-scheduler`, `kube-controller-manager`) need to run *as Pods*, but there's no working Control Plane yet to schedule them. So instead, the kubelet on the Control Plane machine starts them directly from local manifest files. Once running, they show up as read-only "mirror" entries when you run `kubectl get pods`, but you can't edit or delete them through `kubectl` — you have to edit the file on the actual machine.

3. Client-Server Communication Flow

```
kubectl (reads your ~/.kube/config for cluster address + credentials)
   → sends HTTPS request to API Server
   → API Server checks: who are you? are you allowed? does this need
modifying?
   → saved into etcd
   → API Server notifies any component that's "watching" this type of
change
   → Controllers/Scheduler/kubelet react and do their part
   → you can query the current state back at any time with `kubectl
get`
```

The important detail here is the **watch mechanism**: rather than every component repeatedly asking "did anything change yet?", they open a long-lived connection to the API Server and get pushed updates the instant something changes. This is much more efficient than constant polling.

4. Probes

Kubernetes doesn't just start your container and assume it stays healthy forever — the kubelet actively checks on it using up to three types of probes:

Probe	What it's really asking	What happens on failure
Startup	"Has the app finished its slow startup process yet?"	Keeps checking patiently; if it never succeeds within the allowed time, the container is killed and restarted. While this probe is running, Liveness and Readiness checks are paused.
Liveness	"Is the app still functioning, or has it gotten stuck/deadlocked?"	The container is killed and restarted (this is a real fix, not just a warning)
Readiness	"Is the app currently able to handle real user traffic?"	The Pod is temporarily removed from the Service's traffic list — it's NOT restarted, just skipped until it reports ready again

The key distinction to remember: **Liveness = "restart it, it's broken." Readiness = "don't send it traffic yet, but don't touch it."**

```
livenessProbe:
  httpGet: {path: /healthz, port: 8080}
  periodSeconds: 10
  failureThreshold: 3
readinessProbe:
  tcpSocket: {port: 8080}
  periodSeconds: 5
```

Chunk 4: Controllers & Scaling

1. ReplicationController vs ReplicaSet

Both exist to guarantee "N copies of this Pod should always be running" — if a copy dies, a replacement is created automatically. The difference is in how flexible their matching rules are.

Aspect	ReplicationController	ReplicaSet
How it decides which Pods belong to it	Simple equality only (`key=value`)	Simple equality, plus more flexible rules like "in this list of values" or "this label exists"
Still used in practice?	Rarely — considered legacy	Yes, but usually you don't create it directly — a Deployment creates and manages it for you

2. Labels and Selectors

Labels are key-value tags you attach to any object (`env: prod`, `tier: frontend`). They don't have to be unique — many objects can share the same label. **Selectors** are how other objects (Services, ReplicaSets) find the right group of Pods to work with, by matching on those labels.

```
kubectl label pod mypod env=prod
kubectl get pods -l env=prod                    # simple equality
match
kubectl get pods -l 'env in (prod,staging)'      # more flexible, set-
based match
```

3. DaemonSet

Some workloads need to run **exactly once per machine** — not "3 copies somewhere," but literally one copy on every Node (or a specific subset of Nodes). A DaemonSet guarantees this: when a new Node joins the cluster, it automatically gets a copy; when a Node leaves, its copy is cleaned up automatically.

Typical examples: a log-shipping agent, a monitoring agent, or the networking plugin itself — anything that needs a presence on every machine to do its job.

4. Deployments

A Deployment is what you use to manage a stateless application's full lifecycle — not just "keep N copies running" (that's the ReplicaSet's job underneath), but also handling *safe updates* when you change something like the container image version.

There are two ways it can roll out a change:

- **Recreate** — stop everything currently running, then start the new version. Simple to reason about, but causes a real gap where nothing is available.
- **RollingUpdate** (the default) — swap old Pods for new ones gradually, controlled by two settings:
  - `maxSurge`: how many *extra* Pods beyond your normal count are allowed to exist temporarily during the rollout (default 25%)
  - `maxUnavailable`: how many Pods are allowed to be missing/down at once during the rollout (default 25%)

```
Example: 4 replicas, maxSurge=1, maxUnavailable=1

[old][old][old][old]                    → starting point
[old][old][old][old][new]               → 1 new Pod added first
(temporary surge)
[old][old][old][new]                    → 1 old Pod removed
[old][old][old][new][new]               → another new Pod added
... continues one step at a time until only [new][new][new][new]
remain

kubectl rollout status deployment/nginx
kubectl rollout undo deployment/nginx --to-revision=2
```

Chunk 5: Resource Management & Autoscaling

1. Requests vs Limits

Every container can declare two separate numbers for CPU and memory:

- **Request** — "reserve this much for me." The Scheduler uses this number when deciding which Node has enough room for your Pod. It's a guarantee, not a cap.
- **Limit** — "never let me use more than this." This is enforced by the machine itself using Linux cgroups.

What happens if you go over the limit depends on the resource type, and this is an important distinction:

- **Over the CPU limit** → your container is **throttled** (slowed down), not killed. It just gets less CPU time than it wants.
- **Over the memory limit** → your container is **killed immediately** (this shows up as `OOMKilled`, exit code 137) — because unlike CPU, memory can't just be "throttled," so Kubernetes has no choice but to stop it.

```yaml
resources:
  requests: {cpu: "250m", memory: "256Mi"}
  limits:   {cpu: "500m", memory: "512Mi"}
```

This also determines your Pod's **QoS (Quality of Service) class**, which affects who gets evicted first if the Node runs low on resources: Pods where request equals limit (`Guaranteed`) are protected the most; Pods with no requests/limits set at all (`BestEffort`) are the first to be evicted.

2. Metrics Server

A lightweight component that periodically asks every kubelet "how much CPU/memory is actually being used right now?" and makes that data available through a simple API. It only tracks the current snapshot — it doesn't store history, so it's not a replacement for a real monitoring system like Prometheus. Its main consumers are `kubectl top` and the Horizontal Pod Autoscaler.

```
kubectl top nodes
kubectl top pods
```

3. Horizontal Pod Autoscaler (HPA)

Automatically changes how many replicas a Deployment has, based on real usage. It re-checks roughly every 15 seconds and uses this formula:

```
desiredReplicas = ceil( currentReplicas × (currentMetricValue / desiredMetricValue) )
```

In plain terms: *if usage is running at double your target, roughly double your replica count; if usage is at half your target, roughly halve it.* Example: 4 replicas running at 80% CPU, but your target is 50% → `ceil(4 × 80/50) = 7` replicas.

To avoid constantly flapping up and down on small fluctuations, HPA scales up quickly but scales down cautiously, waiting several minutes of sustained lower usage before removing replicas.

```
kubectl autoscale deployment nginx --cpu-percent=50 --min=2 --max=10
```

Chunk 6: Networking & Services

1. Services

Pods are disposable — they get replaced constantly, and each new one gets a new IP address. A **Service** solves this by giving you one stable address that automatically routes to whichever Pods are currently healthy, no matter how many times they get replaced underneath.

Type	How traffic actually gets routed	When you'd use it
ClusterIP (default)	A virtual IP only reachable from inside the cluster; kube-proxy rewrites the destination to a real Pod IP	App-to-app traffic that never needs to leave the cluster
NodePort	Opens the same fixed port (30000-32767) on every worker machine; hitting any machine on that port gets routed to the Service	Quick testing/access without setting up a cloud load balancer
LoadBalancer	Provisions an actual cloud load balancer (like an AWS ELB) in front of a NodePort	Real production traffic coming from the public internet
ExternalName	No routing at all — it's just a DNS shortcut (CNAME) pointing at something outside the cluster	Giving an internal-friendly name to an external system, like a managed database

```
Internet → LoadBalancer → NodePort (any node) → ClusterIP (virtual) →
real Pod IP
```

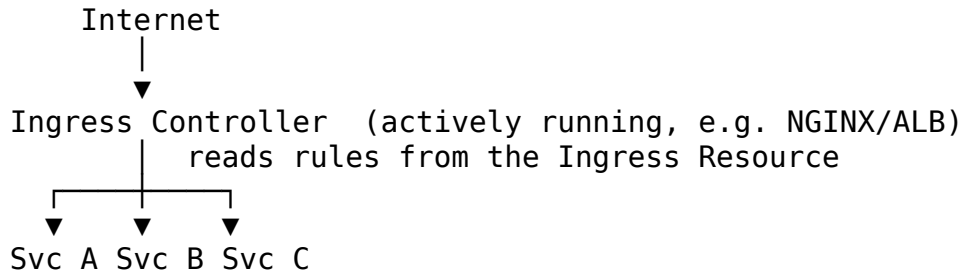
2. Ingress: Resource vs Controller

Running a separate cloud load balancer for every single app gets expensive and messy. **Ingress** solves this by letting you route traffic based on the URL — e.g., `/api` goes to one app, `shop.example.com` goes to another — all through one shared entry point.

There are two separate pieces here, and people often confuse them:

- The **Ingress Resource** is just a YAML file describing your routing rules. On its own, it does absolutely nothing.
- The **Ingress Controller** is the actual running software (like NGINX or AWS's Load Balancer Controller) that watches for Ingress Resources and configures itself to actually enforce those rules. **Without an Ingress Controller installed, your Ingress Resource is inert — Kubernetes doesn't ship one by default.**

```
[Image Placeholder: ingress-flow.png — Internet → Ingress Controller
(reads the rules)
 → routes by hostname/path → correct backend Service → Pods]
```

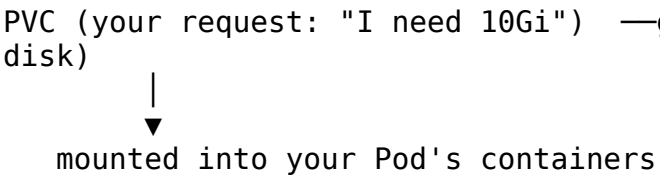


Chunk 7: Storage Architecture

1. Storage Hierarchy

Not all storage in Kubernetes behaves the same way — the key question is: **does this data need to survive the Pod being deleted?**

- **Ephemeral volumes** (like `emptyDir`) — created fresh when the Pod starts, deleted forever when the Pod is removed. Good for temporary scratch space or sharing files between containers in the same Pod.
- **Persistent storage** — survives independently of any one Pod's lifecycle. This uses two connected objects:
  - **PersistentVolume (PV)** — represents an actual piece of storage that exists (like a disk), managed at the cluster level.
  - **PersistentVolumeClaim (PVC)** — your request for storage ("I need 10Gi, read-write by one Pod at a time"). Kubernetes matches your PVC to a suitable PV automatically.



2. Storage Classes

A **StorageClass** defines *how* new storage should be created automatically when someone makes a PVC, instead of an administrator having to manually pre-create disks ahead of time.

- **Static provisioning** — an admin manually creates PVs in advance; PVCs just pick from what's available.
- **Dynamic provisioning** — a PV is created automatically, on the spot, the moment a PVC asks for one, based on a StorageClass. This is the normal approach in cloud environments like EKS.

3. Cloud Storage Basics

	AWS EBS	AWS EFS	GCP Persistent Disk
Type	Block storage (like a private disk)	File storage (a shared network drive)	Block storage
Can multiple Pods write to it at once?	No — one Pod at a time	Yes	Generally no
Typical use	Databases	Shared uploads, shared config, ML datasets	Databases

4. GKE vs EKS Managed Storage

On GKE, dynamic storage provisioning basically works out of the box — there's a default StorageClass ready to go. On EKS, you typically need to install the EBS CSI driver yourself (either as an EKS Add-on or via Helm) before dynamic provisioning will work.

Chunk 8: Configuration & Security (RBAC)

1. ConfigMap vs Secret

Both store configuration data that your Pods can read — as environment variables or as mounted files. The difference is about sensitivity, not mechanics.

	ConfigMap	Secret
Meant for	Ordinary settings (log level, feature flags)	Sensitive data (passwords, API keys, TLS certificates)
How it's stored	Plain text	Base64-encoded — **this is not real encryption**, just a text-safe encoding, easily

	ConfigMap	Secret
		reversible by anyone who can read it
How to actually protect it	N/A	Restrict who can `get/list` Secrets via RBAC, and turn on encryption-at-rest in etcd for real protection

```
kubectl create secret generic db-secret --from-literal=DB_PASS=s3cr3t
```

2. RBAC Architecture

RBAC controls exactly what each person or program is allowed to do. There are four building blocks, and they pair up:

```
Role (permissions, limited to ONE namespace)  ──granted via──▶
RoleBinding (also limited to that namespace)
ClusterRole (permissions, cluster-wide)        ──granted via──▶
ClusterRoleBinding (applies everywhere)
```

A useful detail: a **ClusterRole** can also be handed out through a regular **RoleBinding**, which scopes it down to just one namespace — this is a common pattern for reusing the same permission set across multiple namespaces without redefining it each time.

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {namespace: dev, name: pod-reader}
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: {name: read-pods, namespace: dev}
subjects:
  - {kind: User, name: jane, apiGroup: rbac.authorization.k8s.io}
roleRef: {kind: Role, name: pod-reader, apiGroup:
rbac.authorization.k8s.io}
```

```
kubectl auth can-i list pods --as=jane -n dev
```

3. Human Users vs ServiceAccounts

Kubernetes doesn't actually have a built-in concept of "user accounts" for humans — instead, it trusts whatever identity your certificate or token proves (managed outside Kubernetes, e.g., by AWS IAM on EKS). **ServiceAccounts**, on the other hand, are real objects that live inside Kubernetes, and they exist specifically so that a running Pod can authenticate itself to the API Server (for example, a CI/CD job that needs to create other Pods).

On EKS specifically, you can link a ServiceAccount to a real AWS IAM Role (a feature called **IRSA** — IAM Roles for Service Accounts), so a Pod can securely call AWS APIs (like reading from S3) without needing to store any AWS credentials inside the cluster.

Chunk 9: Helm Package Management

1. Helm Concepts

Helm is a package manager for Kubernetes — similar in spirit to `apt` or `npm`, but for deploying whole applications made of many Kubernetes objects at once.

- **Chart** — a packaged, reusable template describing how to deploy an application, with configurable defaults.
- **Repository** — a hosted collection of Charts you can pull from.
- **Release** — one actual, running installation of a Chart. You can install the same Chart multiple times under different Release names (e.g., two separate WordPress installs in the same cluster).

2. Helm Architecture (v3)

Earlier versions of Helm (v2) needed a component called **Tiller** running permanently inside your cluster with broad permissions — this was a well-known security weak point, since anyone who could reach Tiller effectively had wide control over the cluster.

Helm v3 removed Tiller entirely. Now the Helm command-line tool talks directly to the API Server using your own kubeconfig and your own RBAC permissions — there's no separate in-cluster component to secure. Helm keeps track of what it has installed (a "Release") by storing that history as a Secret object in the relevant namespace.

3. Core Commands

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

helm install my-release bitnami/nginx -n web --create-namespace
helm upgrade my-release bitnami/nginx --set replicaCount=3
helm upgrade --install my-release bitnami/nginx    # install if
missing, upgrade if it exists
helm rollback my-release 1
helm uninstall my-release

helm list -A
helm status my-release
helm history my-release
helm template my-release bitnami/nginx   # preview the rendered YAML
without installing anything
```